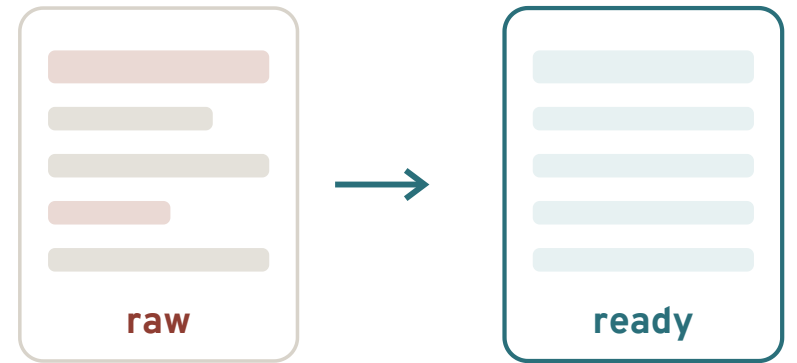


DATA PREPARATION AND REPRODUCIBLE WORKFLOWS

Dr. Rebin Muhammad



MAKE THE DATA USABLE, HONEST, AND REPRODUCIBLE

Week 2 agenda

- 1 What makes data “ready” for machine learning?
- 2 Features, response, and data structure
- 3 Data quality and missing values
- 4 Categorical variables
- 5 Scaling and transformations
- 6 Train/test split and leakage
- 7 Reproducible preprocessing

A MODEL IS ONLY AS GOOD AS THE DATA WE GIVE IT

From a statistical learning point of view, we need to understand the input variables in order to understand or predict an output.

From a programming point of view, the way we represent those variables can greatly affect performance.

Same information, different representation — and a very different model.

FAVORITE COLOR, CODED AS NUMBERS

Red = **1**

Blue = **2**

Green = **3**

Green is not “greater than” Blue. Colors have no order.

TREAT EACH COLOR AS ITS OWN YES / NO

A customer whose favorite color is Red:

is_red = 1 is_blue = 0 is_green = 0

ROWS, COLUMNS, AND THE QUESTION

age	education	occupation
34	BA	nurse
51	HS	teacher

● features / predictors ● response / target

Each row is one observation. Example: predict income category from age, education, occupation.

Prediction

Can we accurately predict the outcome?

Inference

Can we understand how variables are associated with the outcome?

Supervised

We have a target. Example: income above or below \$50K.

Unsupervised

No target. Example: group customers by behavior.

Bad data preparation can damage prediction and make interpretation misleading.

RAW DATA IS RARELY MODEL-READY

Real datasets often contain blank entries, inconsistent labels, numbers on different scales, outliers, and duplicate rows. These are not just small mistakes. They can change the model. Before modeling, we inspect.

age	income	gender
-3	52,000	Male
34	blank	M

Impossible age, a blank income, three spellings of the same gender, 200 hours in a week, and a ZIP that lost its leading zero because it was stored as a number.

DEFINITION

WHAT DATA PREPARATION MEANS

Checking and transforming raw data so it can be used responsibly in a model.

- 1 Missing values
- 2 Wrong data types
- 3 Categorical variables
- 4 Scaling
- 5 Transformations
- 6 Feature engineering
- 7 Train/test separation
- 8 Leakage prevention

DATA QUALITY QUESTIONS

What does each row represent?

What does each column mean?

Are values missing?

Are categories consistent?

Are units clear?

Are there impossible values?

Was the data collected ethically?

MISSING DATA

Common approaches

remove rows

remove columns

fill with mean / median / mode (imputation)

create a missingness indicator

use domain knowledge

study hours	hours missing?	
6	0	
—	1	

No single method is always correct.

MISSINGNESS INDICATOR

Also create **Study Hours Missing?** Yes = 1, No = 0.
The fact it was missing can be a signal.

DOMAIN KNOWLEDGE

A missing garage size may mean the home has no garage. Record **Garage size = 0** instead of filling the average. Missing is not always “unknown.”

WRONG DATA TYPE

The column is stored in a form that does not match what the variable actually represents. The value itself may be fine. The computer is reading it the wrong way.

Age as text

“21”, “35”, “42” instead of numbers – you cannot take a mean of a string.

Date as text

“09/06/2026” instead of a date type – you cannot compute “days until.”

ZIP as a number

It is an identifier, not a quantity. 02138 becoming 2138 is the computer dropping a leading zero.

Yes / No mixed

“Yes”, “Y”, 1, and True in the same column look like four different things.

CATEGORICAL VARIABLES

Many models cannot directly use text categories.

occupation = “teacher”

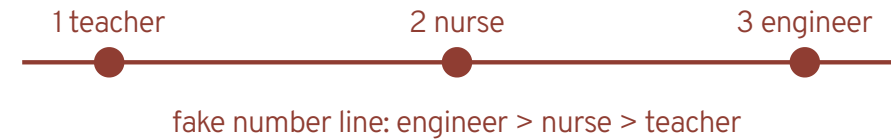
occupation = “nurse”

occupation = “engineer”

We must convert categories into a numeric representation. The danger is inventing an order that is not real.

DO NOT DO THIS

teacher = 1 nurse = 2 engineer = 3



NUMBERS THAT ARE REALLY CATEGORIES

ZIP code, student ID, course number, hospital code.
They look numeric, but should not always be treated as continuous.

WHY ONE-HOT ENCODING HELPS

It avoids fake order. Instead of $\text{teacher} < \text{nurse} < \text{engineer}$, we ask yes/no for each category. Each row has **exactly one 1**.

occupation	teacher	nurse
teacher	1	0
nurse	0	1

This creates columns such as `occupation_teacher`, `occupation_nurse`, `occupation_engineer`.

A teacher is not “less than” a nurse. They are simply different. This is the standard way to represent unordered categories for machine-learning models.

SCALING

Scaling puts numeric features on comparable ranges so they contribute more equally during training.



For a distance-based model such as KNN, income can dominate simply because its numbers are much larger. Two people 30 years apart in age, but \$1,000 apart in income:

$$d = \sqrt{[(60-30)^2 + (41000-40000)^2]} \approx 1000$$

The 30-year age gap barely registers. Scaling keeps the model from giving extra importance to a variable just because of its units.

COMMON SCALING METHODS

STANDARDIZATION

Subtract the mean, divide by the standard deviation. The result is in units of “how unusual.”

$$z = (x - \mu) / \sigma = (80 - 40) / 20 = 2$$

Here age is 80, the class mean is 40, and $\sigma = 20$. A z of 2 means two standard deviations above typical – unusual, not impossible. Values beyond about -2 or 2 are a useful flag for extremes.

MIN-MAX SCALING

Move values into a range, often 0 to 1.

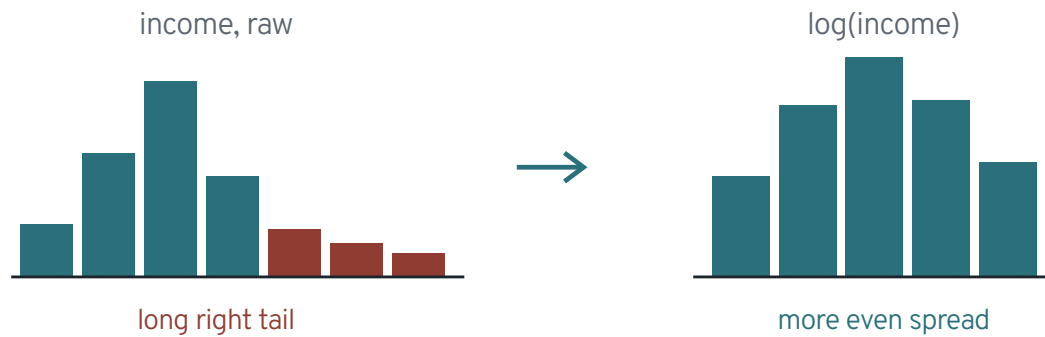
$$x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}}) = (50 - 18) / (80 - 18) \approx 0.52$$

Age 50, with min 18 and max 80. **Sensitive to outliers:** one extreme can squash typical values toward 0. **Needs fixed bounds:** a new point outside the original min and max falls outside $[0,1]$ unless you recompute.

Use scaling when the model depends on distance, gradients, or magnitude.

TRANSFORMATIONS

Convert raw data into a cleaner numerical form so a model can learn from it. Transformations can make patterns easier to see.



Suppose we are predicting house price.

Skew. If income is very skewed, use $\log(\text{income})$ so extremely large incomes have less influence.

Groups. Instead of exact age, create groups such as 18–29, 30–49, and 50+.

Interaction. Square footage may matter more in some neighborhoods: $\text{sqft} \times \text{neighborhood}$.

FEATURE ENGINEERING

Creating better input variables. Representing data well is one of the main tasks in machine learning.

birth year → age

The year itself is less useful than how old the person is now.

date → day of week

Monday traffic is not the same as Saturday traffic.

income and household size → income per person

The same income means different things for 1 person vs. 5.

hours studied × attendance

An interaction: studying may help more if the student also shows up.

TRAIN / TEST SPLIT

Train on one part of the data. Test on a separate part the model has not seen. That is how we evaluate performance on new, real-world data.

training set – the model may look here

test – hold out

Generalization: can the model predict new data, or did it only memorize the training set?

Overfitting: accuracy on the training data can look unrealistically high.

Leakage: keeping the test set separate stops test information from shaping training or feature engineering.

DATA LEAKAGE

Leakage happens when information from the test set, or from the future outcome, sneaks into training.

WRONG

all rows → mean of everyone, including test → fill blanks → then split

The model never saw the test labels – but the test rows still helped invent the mean. That is still cheating.

RIGHT

split first → mean from training only → fill training → use that same mean on test

The test set is touched only after the recipe is already fixed.

Leakage makes the model look better than it really is. It is one of the most serious mistakes in machine learning.

EXAMPLE 1

PREDICTING WHETHER A STUDENT WILL PASS

Final grade 60% or higher = Pass. Below 60% = Fail.

attendance	homework	midterm
90%	88	74
62%	55	48

DO NOT USE AS A FEATURE

Final grade already determines Pass/Fail. Including it gives the model the answer.

USE INSTEAD

Features: attendance, homework, midterm

Target: Pass/Fail

Split into training and test. The model learns from training data, then predicts Pass/Fail for test students using only attendance, homework, and midterm.

EXAMPLE 2

WHY WE SPLIT BEFORE SCALING

Training study hours: 2, 4, 6, 8. One test student studied 20 hours. If that 20 is used to set the scale, it changes the training numbers.

hours	set	min-max using everyone min 2, max 20
2	train	0.00
8	train	0.33

$x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$. Including the test 20 compresses the training 8 from 1 down to 0.33. Test information has then shaped the training data.

CORRECT PROCESS

Split first. Learn min and max from training only. Scale training with those values. Use the **same** training values on the test set. A test value outside [0,1] is allowed – that is honest.

THE BIG RULE

Anything learned from the data must be learned from the training set only. Then apply the same transformation to the test set.

This includes:

means

standard deviations

category levels

imputation values

scaling values

CORRECT WORKFLOW

STEP 1

Split data into training and test

STEP 2

**Learn preprocessing from
training data**

STEP 3

**Apply preprocessing to training
data**

STEP 4

**Apply the same preprocessing to
test data**

STEP 5

Train the model

STEP 6

Evaluate on test data

The test set is touched only after the preprocessing recipe is already fixed.

NOT EVERY CATEGORY IS UNORDERED

One-hot is for labels with no rank. If the categories really do have an order, a number can be the honest representation.

NOMINAL – NO ORDER

Color, occupation, ZIP, hospital code.

Red / Blue / Green is not a ladder. Use one-hot.

ORDINAL – REAL ORDER

Education: HS $\rightarrow$ BA $\rightarrow$ MA $\rightarrow$ PhD

Coding 1, 2, 3, 4 is fair here. A PhD really is further along that path than a BA.

Ask first: does a bigger number mean something real? If no, one-hot. If yes, an ordered code is allowed – still learned from the training set only.

WE DO NOT ALWAYS SCALE

SCALE THESE

KNN, other distance methods, neural nets, anything using gradients.

These models care about the size of the numbers. Income in dollars will drown age in years unless you scale.

USUALLY SKIP THESE

Decision trees and random forests.

A tree asks “is income > 50,000?” It never computes a distance, so the units do not steal the show.

If you are unsure which model you will try, scaling the numeric features is the safer default – but know why you are doing it.

QUESTIONS

1. Will this patient be readmitted?

The file includes notes from the second hospital stay. Can we use them? Why or why not?

2. Will this customer cancel next month?

Someone offers the column “date they cancelled.” Is that a feature or the answer?

3. House prices, 2018-2024

If we shuffle all years and split at random, the test set can contain 2019 houses while the training set already saw 2024 prices. What should we do instead?

REPRODUCIBILITY

A reproducible workflow means someone else can follow your steps and get the same result.

clear code

each cleaning step is written down, not done by hand in a spreadsheet and forgotten

fixed random seed

the train/test split does not change every time you rerun

documented assumptions

why garage size missing became 0, why ZIP is a category

saved preprocessing

the same means, scales, and category levels can be reused

organized files

raw data, cleaned data, and code live in obvious places

clear train/test process

someone else can see that the test set stayed unseen